

TP — Application CodeIgniter 4 : Gestion de Bibliothèque

Contexte

Vous allez développer une application web complète de gestion de bibliothèque avec CodeIgniter 4. L'application permettra de gérer un catalogue de livres, d'effectuer des emprunts et des retours, et de garantir la sécurité et l'intégrité des données.

Ce TP mobilise l'ensemble des notions suivantes : routage, modèles, validation, CSRF, Query Builder, templating et pagination.

Partie 1

1.1 Créer une base de données MySQL nommée **bibliotheque** en UTF-8.

1.2 Créer une table **livres** contenant les champs suivants : identifiant auto-incrémenté, titre, auteur, ISBN (unique), année de publication, catégorie, résumé, nom de fichier de couverture, statut (disponible / prêté), et les timestamps de création/modification.

1.3 Créer une table **emprunts** permettant de tracer chaque prêt : identifiant, référence au livre, nom de l'emprunteur, date d'emprunt et date de retour (nullable).

1.4 Configurer la connexion à la base dans le fichier approprié de CodeIgniter.

Partie 2

Définir l'ensemble des routes nécessaires à l'application.

2.1 Une route pour afficher la liste de tous les livres (page d'accueil).

2.2 Une route pour afficher la fiche détaillée d'un livre, en passant son identifiant dans l'URL.

2.3 Deux routes pour l'ajout d'un livre : une pour afficher le formulaire (GET), une pour traiter l'envoi (POST).

2.4 Une route POST pour supprimer un livre.

2.5 Deux routes POST pour les actions de prêt et de retour d'un livre, avec l'identifiant du livre en paramètre d'URL.

Justifier le choix de la méthode HTTP pour chaque route liée à une action de modification.

Partie 3

Créer le modèle correspondant à la table **livres**.

3.1 Déclarer la table, la clé primaire, les champs autorisés et activer les timestamps automatiques.

3.2 Définir les règles de validation suivantes directement dans le modèle :

- Le titre est obligatoire et doit comporter au moins 3 caractères.
- L'auteur est obligatoire.
- L'ISBN est obligatoire et doit être **unique en base de données**.
- L'année est obligatoire.

Rédiger des messages d'erreur explicites en français pour chaque règle.

3.3 Ajouter une méthode de validation métier pour s'assurer que l'année de publication n'est pas dans le futur.

3.4 Ajouter une méthode de recherche permettant de filtrer les livres par mot-clé (sur le titre) et optionnellement par catégorie.

3.5 Ajouter une méthode retournant les livres paginés (10 résultats par page).

Partie 4

Créer le modèle correspondant à la table **emprunts**.

4.1 Déclarer la table et les champs autorisés.

4.2 Ajouter une méthode retournant le dernier emprunt enregistré pour un livre donné (le plus récent selon la date d'emprunt).

Partie 5

Créer le contrôleur principal de gestion des livres.

5.1 — Liste (index) : Récupérer les livres avec pagination. Si des paramètres de recherche sont présents dans l'URL (mot-clé ou catégorie), utiliser la méthode de recherche à la place. Transmettre les données à la vue correspondante.

5.2 — Détail : Récupérer un livre par son identifiant. Si le livre n'existe pas, déclencher une page 404. Récupérer également le dernier emprunteur via **EmpruntModel** et transmettre les deux à la vue.

5.3 — Formulaire : Afficher simplement le formulaire d'ajout.

5.4 — Enregistrement (store) :

- Récupérer les données POST.
- Valider l'année manuellement (règle métier).
- Si une image de couverture est envoyée, vérifier que c'est bien un fichier image (jpeg, png ou webp) et que sa taille ne dépasse pas 2 Mo. En cas d'erreur, réafficher le formulaire avec un message. Sinon, enregistrer le fichier dans **public/uploads/**.
- Utiliser le modèle pour insérer les données ; en cas d'échec de validation, réafficher le formulaire avec les erreurs.

- En cas de succès, rediriger vers la liste avec un message flash de confirmation.

5.5 — Suppression : Supprimer le livre et rediriger avec un message flash.

Partie 6

Créer le contrôleur gérant les mouvements de livres.

6.1 — Prêt : Vérifier que le livre existe et qu'il est disponible. Si ce n'est pas le cas, rediriger avec un message d'erreur. Sinon, enregistrer un nouvel emprunt avec le nom de l'emprunteur et la date du jour, puis mettre à jour le statut du livre à "prêté".

6.2 — Retour : Retrouver l'emprunt actif (sans date de retour) pour ce livre, renseigner la date de retour, et remettre le statut du livre à "disponible".

Partie 7

Créer l'ensemble des vues en utilisant le système de templating de CodeIgniter (layout + sections).

7.1 — Layout principal : Créer un fichier de mise en page commune incluant la barre de navigation, la zone d'affichage des messages flash (succès et erreur), et la zone de contenu dynamique.

7.2 — Vue catalogue :

- Afficher un formulaire de recherche (champ texte + liste de catégories).
- Afficher un tableau listant tous les livres avec : titre (lien vers la fiche), auteur, année, statut coloré (vert = disponible, rouge = prêté).
- Pour chaque livre disponible : afficher un formulaire inline pour saisir le nom de l'emprunteur et un bouton "Prêter".
- Pour chaque livre prêté : afficher uniquement le bouton "Retourner".
- Afficher un bouton "Supprimer" avec confirmation JavaScript.
- Afficher les liens de pagination si nécessaire.

7.3 — Vue détail : Afficher toutes les informations du livre ainsi que le nom et la date d'emprunt du dernier emprunteur (si disponible).

7.4 — Formulaire d'ajout :

- Champs : titre, auteur, ISBN, année, catégorie (liste déroulante), résumé, couverture (upload).
 - Afficher les messages d'erreur de validation sous chaque champ concerné.
 - Utiliser `old()` pour repopuler les champs en cas d'erreur.
 - Inclure le jeton CSRF.
 - Limiter l'année maximale à l'année en cours côté client.
-

Partie 8

8.1 Activer le filtre CSRF globalement pour toutes les requêtes.

8.2 S'assurer que tous les formulaires POST contiennent le jeton CSRF.

8.3 Utiliser `esc()` sur toutes les données affichées dans les vues pour prévenir les failles XSS.

Critères d'évaluation

- Fonctionnement correct des opérations CRUD
- Respect de la validation et affichage des erreurs
- Protection CSRF active et fonctionnelle
- Gestion correcte du statut (prêt / retour / désactivation du bouton)
- Pagination visible dès 11 livres
- Validation du fichier image (type + taille)
- Code organisé selon le pattern MVC
- Vues propres, utilisant le layout et `esc()`